

Browser-Based Email Sending Feasibility Report

Exploration summary for adding browser automation while preserving inbox reliability

Short answer: I explored the browser-bot idea and it is technically possible. My recommendation is a hybrid approach: keep IMAP/app passwords for inbox receiving, and add browser-based sending as an optional sending mode.

Executive Summary

The client request is to send emails through a hidden or automated browser instead of SMTP. The goal is to make the app behave more like normal webmail: log in to Gmail, Yahoo, AOL, etc., open compose, fill in the recipient, subject, and message, and send from the browser.

After reviewing the current project structure, the safest path is not to replace the whole mail system. The current application already has working SMTP and IMAP flows. Browser automation should be added only as a new sending option, while inbox downloading and reply detection should continue through IMAP.

- Browser automation can be used for sending.
- IMAP with app passwords should stay responsible for receiving replies.
- Each account should have its own saved browser profile/cookies.
- Only a small number of browsers should run at the same time.

1. Lowest Possible Specs For 100 Accounts

The lowest-spec design depends on a key distinction: storing 100 accounts is easy, but running 100 live browser sessions is not. A browser is much heavier than an SMTP connection, so the system should queue accounts and run only a small worker pool.

Scenario	Suggested Specs	Notes
100 accounts stored, 1 browser active	2 vCPU, 4 GB RAM	Lowest practical setup. Slowest, but cost-efficient.
100 accounts stored, 2-3 browsers active	2-4 vCPU, 8 GB RAM	Recommended baseline for smoother operation.
5-8 active browser workers	4-8 vCPU, 16 GB RAM	Higher throughput, but noticeably heavier.
100 active browser sessions	Not recommended	Too heavy and likely to increase account challenges.

Recommendation: for the lowest possible setup, use 1-2 active browser workers and queue the remaining accounts. This keeps RAM and CPU usage low. It will be slower than SMTP, but it is the practical tradeoff for low infrastructure cost.

For the current Gmonster app, this means browser concurrency should be controlled separately from the existing SMTP thread limit. SMTP threads are lightweight; browser workers are not.

2. Solving The Inbox Issue

The main concern is understandable: browser login uses the normal account password, while IMAP usually needs 2FA and an app password. The important point is that these are separate login methods and they can normally coexist.

Purpose	Method	Credential / Session
Browser sending	Webmail login	Normal password + 2FA, then saved browser session
Inbox download	IMAP	App password
Existing SMTP sending	SMTP	App password

Important clarification: having 2FA and an app password does not mean browser login stops working. The user can still log in through the browser with the normal password and complete 2FA. IMAP can continue using the app password.

Recommended inbox solution:

- Keep the current IMAP/app-password inbox download flow.
- Add browser sending as an optional mode only.
- Store browser cookies/session data separately from IMAP credentials.
- Do not use browser scraping as the primary inbox solution unless IMAP/OAuth is not available.

A longer-term improvement would be to support official APIs such as Gmail API or Microsoft Graph for inbox and sending. That is cleaner than browser inbox scraping, but it is a bigger integration project.

3. CAPTCHA, 2FA, And Cookie Storage

CAPTCHAs and login challenges cannot be fully avoided if a provider decides the login looks risky. The practical solution is to reduce the triggers that cause challenges and to store legitimate sessions after the user completes login.

Recommended flow:

- Open a visible browser during account setup.
- Let the user log in manually and complete 2FA or CAPTCHA if shown.
- After successful login, save the browser profile/cookies for that account.
- Reuse the same saved session for future sending.
- Use one browser profile per account, for example: data/browser_profiles/account_hash/.
- Keep the same proxy/IP per account where possible.
- Avoid aggressive proxy rotation and repeated login attempts.
- If a challenge appears later, pause that account and ask the user to resolve it manually.

Compliance note: I do not recommend CAPTCHA bypass services, stealth fingerprint hacks, or automated challenge solving. They increase account risk and make the system fragile. The reliable approach is stable sessions, low concurrency, and manual resolution when providers request verification.

Recommended Implementation Direction

The best product design is a hybrid sending mode:

- Keep the existing SMTP sender as one option.
- Add a new Browser Sender option for accounts that should send through webmail.
- Keep IMAP receiving unchanged for inbox, reply checks, and follow-up logic.
- Add browser session onboarding so each account can be logged in once and reused.
- Default browser concurrency to 1-2 workers on low-spec systems.

References

[Google Account Help - Sign in with app passwords](https://support.google.com/accounts/answer/185833)

<https://support.google.com/accounts/answer/185833>

[Gmail Help - Check Gmail through other email platforms](https://support.google.com/mail/answer/7126229)

<https://support.google.com/mail/answer/7126229>

[Playwright - Authentication and saved browser state](https://playwright.dev/python/docs/auth)

<https://playwright.dev/python/docs/auth>

[Playwright - Persistent browser context](https://playwright.dev/python/docs/api/class-browsertype#browser-type-launch-persistent-context)

<https://playwright.dev/python/docs/api/class-browsertype#browser-type-launch-persistent-context>

[Gmail API - Sending email](https://developers.google.com/gmail/api/guides/sending)

<https://developers.google.com/gmail/api/guides/sending>

[Gmail API - Push notifications](https://developers.google.com/workspace/gmail/api/guides/push)

<https://developers.google.com/workspace/gmail/api/guides/push>

[Microsoft Graph - user: sendMail](https://learn.microsoft.com/en-us/graph/api/user-sendmail)

<https://learn.microsoft.com/en-us/graph/api/user-sendmail>

Final Recommendation

I recommend proceeding with the hybrid model: browser-based sending as an optional transport, IMAP/app-password receiving for inbox, saved browser sessions per account, and low browser concurrency by default. This is the most practical way to meet the client request while keeping the product stable.